Introdução à Computação
arrays e strings

Alexandre Rademaker

Array I

src/scores0.c

```c
#include <stdio.h>

int main(void) {

    int score1 = 72;
    int score2 = 73;
    int score3 = 33;

    printf("Average: %d\n", (score1 + score2 + score3) / 3.0);
}
```

Nomes distintos para cada posição na memória! Não necessariamente alocados de forma contínua! Podemos simplificar?

Array II

Isto torna o código mais elegante?

src/scores1.c

```c
#include <stdio.h>

int main(void) {

  int scores[] = {1, 2, 3};

  printf("Average: %f\n", (scores[1] + scores[2] + scores[3]) / 3.0);
}
```

Array III

endereço	valor	
0x10000	?	
0x10004	72	← score1
0x10008	?	
0x10012	?	
0x10016	73	← score2
0x10020	?	
0x10024	?	
0x10028	33	← score3
0x10032	?	
0x10036	?	
0x10040	?	

endereço	valor	
0x10000	?	
0x10004	72	← scores
0x10008	73	
0x10012	33	
0x10016	?	
0x10020	?	
0x10024	?	
0x10028	?	
0x10032	?	
0x10036	?	
0x10040	?	

caracteres e inteiros I

src/hi0.c

```c
#include <stdio.h>

int main(void) {

  char c1 = 'H';
  char c2 = 'I';
  char c3 = '!';

  printf("%c%c%c\n", c1, c2, c3);
}
```

```
> clang hi0.c
> ./a.out
HI!
```

caracteres e inteiros II

src/hi1.c

```c
#include <stdio.h>

int main(void) {

  char c1 = 'H';
  char c2 = 'I';
  char c3 = '!';

  printf("%i %i %i\n", c1, c2, c3);
  return 0;
}
```

```
> ./a.out
72 73 33
```

Os `char` são armazenados como inteiros (códigos ASCII).

caracteres e inteiros III

`printf` fez uma conversão implícita, mas também podemos converter explicitamente.

```
printf("%i %i %i\n", (int) c1, (int) c2, (int) c3);
```

Mas conversões podem perder informação, por exemplo, de `float` para `int`.

Strings I

Representamos strings como arrays de caracteres (cada um com 8 bits ou um byte), um tipo que na verdade não é primitivo em C.

src/hi2.c

```
#include <stdio.h>

int main(void) {

  char a[] = "HI!";
  char b[] = {'H', 'I', '!', '\0'};

  printf("%s\n", s);
}
```

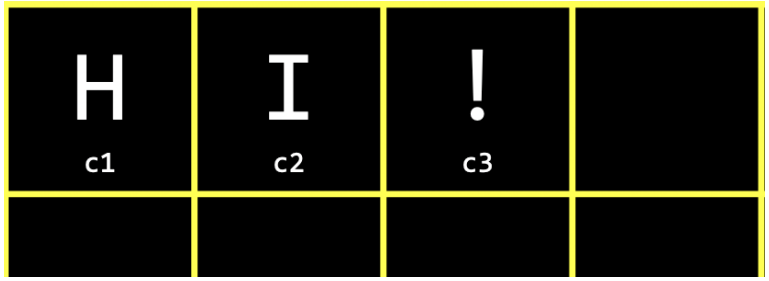
Strings II

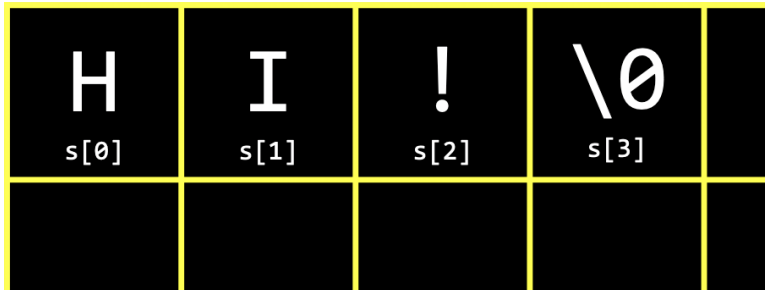
src/hi3.c

```c
#include <stdio.h>

int main(void) {

  char s[] = "HI!";
  printf("%i %i %i\n", s[0], s[1], s[2]);
}
```

```
> ./a.out
72 73 33
```

Strings III





Em C, strings terminam com o char `'\0'` ou inteiro `0`, ou um byte com todos os bits 0, marcando a terminação da string. Este é o chamado 'NUL'. Então precisamos sempre de um byte adicional.

Outros tipos de dados de tamanho fixo não precisam de tal mecanismo. Mas bibliotecas que oferecem representação de variadas precisões numéricas podem usar também tal recurso.

Strings IV

src/hi4.c

```c
#include <stdio.h>

int main(void) {

    char s[] = "HI!";
    printf("%i %i %i %i\n", s[0], s[1], s[2], s[3]);
}
```

```
> ./a.out
72 73 33 0
```

Strings V

src/hi5.c

```c
#include <stdio.h>

int main(void) {

  char s[] = "HI!";
  char t[] = {'B','y','e', '!'};

  // what is wrong?
  for(int i = 0; i < 4; i++)
    printf("loc s[%d] = %c %p\n", i, s[i], &s[i]);
  for(int i = 0; i < 4; i++)
    printf("loc t[%d] = %c %p\n", i, t[i], &t[i]);

  printf("%s and %s \n", t, s);
}
```

Strings VI

s[0]	H
s[1]	i
s[2]	!
s[3]	\0

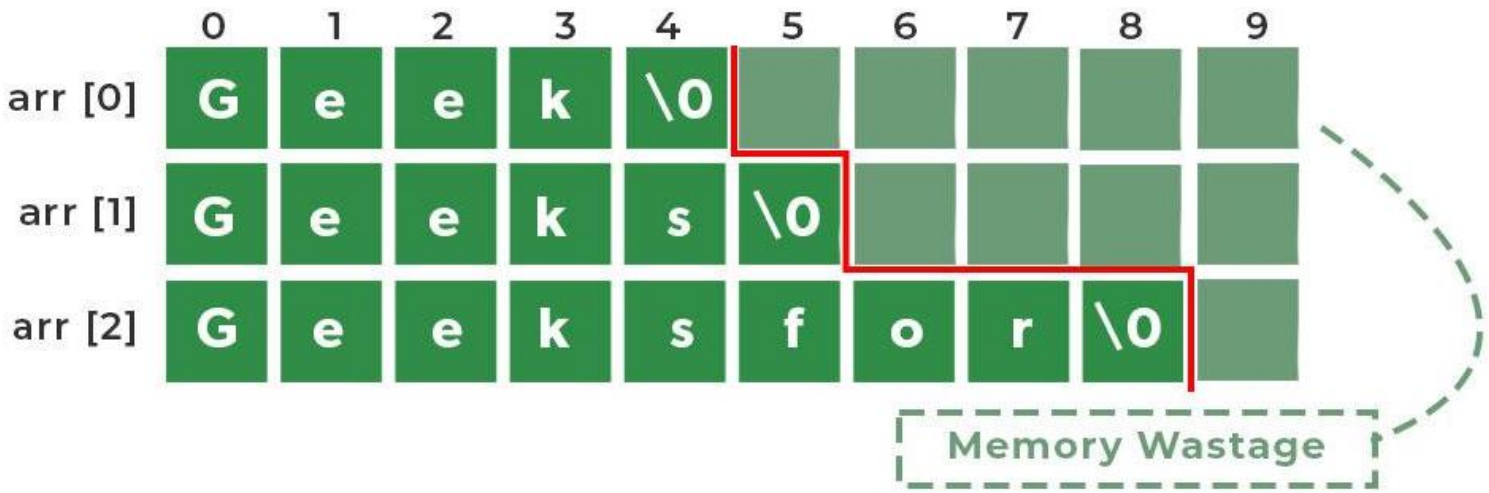
t[0]	B
t[1]	y
t[2]	e
t[3]	!

Strings VII

Um array de strings ou um array de array de caracteres.

src/hi6.c

```
1  #include <stdio.h>
2
3  int main(void) {
4
5    char arr[3][10] = {"Geek", "Geeks", "Geekfor"};
6
7    for (int i = 0; i < 3; i++)
8      {
9        printf("%s\n", arr[i]);
10     }
11 }
```

Strings VIII

Na memória, este array está armazenado da seguinte forma.